

VIT – Vidyalkar Institute of Technology
School of Computer Science and Engineering

INDUSTRIAL INTERNSHIP REPORT

on

PassVault – Secure Password Manager
using Python & PostgreSQL

Prepared by	Organisation
Nisarg Ambawade	upskill Campus / UCT
Industrial Internship Program	UniConverge Technologies Pvt Ltd
Academic Year: 2025–26	Duration: 6 Weeks

 Cybersecurity · Secure Software Development · Cloud Database Integration

Executive Summary

This report documents the design, development, and evaluation of PassVault — a cybersecurity-focused, desktop-based password management system developed as part of a six-week Industrial Internship program facilitated by upskill Campus, The IoT Academy, and UniConverge Technologies Pvt Ltd (UCT). The project was executed in fulfilment of the internship requirement under VIT.

PassVault addresses a critical gap in personal digital security by providing encrypted credential storage, secure master-password authentication, configurable strong password generation, and seamless cloud database integration through a PostgreSQL instance hosted on Neon DB. The application is implemented in Python, leveraging the CustomTkinter library for a modern dark-themed graphical user interface, the cryptography library for Fernet symmetric encryption, psycpg2 for database connectivity, and python-dotenv for environment-variable-based secrets management.

The report covers the problem statement, comparative analysis of existing solutions, system architecture, interface design, performance testing, technical learnings, and future enhancement roadmap. All artefacts — source code, documentation, and this report — are maintained on GitHub and are suitable for academic submission, internship evaluation, and portfolio demonstration.

Preface

This report is the culmination of six weeks of focused effort during the Industrial Internship program organised by upskill Campus in collaboration with The IoT Academy and UniConverge Technologies Pvt Ltd (UCT). The internship was structured to bridge the gap between academic learning and industry-level software engineering practice, with each intern assigned a real-world problem statement to research, design, and implement.

My assigned problem statement was to develop PassVault — a Secure Password Manager using Python and PostgreSQL. The project required me to apply and extend my knowledge of secure software development, cryptographic principles, cloud database integration, and desktop GUI design — all within a production-oriented development workflow.

Over the six-week period, I gained significant hands-on exposure to encryption workflows, PostgreSQL cloud deployment, modular Python architecture, and the Git/GitHub version-control ecosystem. The process of designing, debugging, and iteratively improving PassVault gave me a realistic appreciation of the challenges involved in building secure, production-ready software.

I extend my sincere gratitude to the mentors and team at UniConverge Technologies Pvt Ltd for providing this opportunity, for their technical guidance, and for the industry-relevant problem statement. I also thank upskill Campus and The IoT Academy for the administrative support and learning infrastructure that made this internship programme possible.

To my peers and juniors who undertake this programme in future — approach every problem with curiosity, document your failures as carefully as your successes, and treat the internship as an opportunity to build habits that will define your engineering career.

Nisarg Ambawade

VIT – Vidyalanakar Institute of Technology

Industrial Internship Program | upskill Campus / UCT

Table of Contents

Executive Summary	2
Preface	3
1. Introduction	5
1.1 About UniConverge Technologies Pvt Ltd	5
1.2 About upskill Campus	6
1.3 Objectives of the Internship	6
2. Problem Statement	7
3. Existing and Proposed Solution	8
4. Proposed Design / Model	10
4.1 High-Level Architecture Diagram	10
4.2 Low-Level Flow Diagram	11
4.3 Application Interfaces & Screenshots	12
5. Performance Test	15
5.1 Test Plan and Test Cases	15
5.2 Test Procedure	16
5.3 Performance Outcome	16
6. My Learnings	17
7. Future Work Scope	18
8. Conclusion	19
References	20
Glossary	21

Chapter 1: Introduction

1.1 About UniConverge Technologies Pvt Ltd

UniConverge Technologies Pvt Ltd (UCT) is a technology and digital transformation company established in 2013, headquartered in India. UCT specialises in delivering Industrial IoT platforms, smart factory automation, cloud computing solutions, cybersecurity tools, and machine learning-powered predictive maintenance systems. The company serves clients across manufacturing, agriculture, smart cities, and utilities sectors.

Key product lines and service areas include:

- UCT Insight — A Java and ReactJS-based Industrial IoT platform supporting MQTT, CoAP, HTTP, Modbus TCP, and OPC-UA protocols, enabling cloud and on-premises deployments with dashboarding, alerting, analytics, and third-party integrations (Power BI, SAP, ERP).
- FactoryWatch — A SaaS-model smart factory platform for OEE monitoring, predictive maintenance, and digital twin capabilities at modular scale.
- LoRaWAN Solutions — Early adoption and deployment of long-range, low-power wide-area network (LPWAN) technology for agriculture, smart cities, and utility metering.
- Predictive Maintenance — Industrial machine-health monitoring using embedded systems, IIoT sensors, and machine learning to calculate remaining useful life (RUL) of critical assets.

1.2 About upskill Campus (USC)

upskill Campus (USC) is a career development and EdTech platform that delivers personalised professional education and executive coaching in a scalable, affordable, and outcome-measurable format. USC operates in collaboration with The IoT Academy — the EdTech division of UCT — which runs executive certification programmes in partnership with EICT Academy, IIT Kanpur, IIT Roorkee, and IIT Guwahati.

The internship programme, co-facilitated by USC, UCT, and The IoT Academy, offers students hands-on exposure to real-world engineering problems with industry mentorship, structured project delivery, and verifiable outcomes. USC targets upskilling one million learners over the next five years.

1.3 Objectives of the Internship

The objectives of this six-week industrial internship programme were:

1. Gain practical, hands-on experience in industry-standard software development practices.
2. Analyse and propose solutions to real-world engineering and cybersecurity problems.
3. Design, implement, and test a functional, deployable software application within a defined timeline.
4. Develop proficiency in professional tools including Git/GitHub, PostgreSQL, and Python ecosystem libraries.
5. Strengthen employability through project portfolio development, technical documentation, and structured reporting.

Chapter 2: Problem Statement

The proliferation of digital services has created an unprecedented challenge in personal credential management. The average internet user maintains accounts across dozens of platforms, each requiring unique and complex authentication credentials. In practice, however, a significant proportion of users continue to rely on dangerously insecure credential storage methods.

2.1 The Security Gap in Current Practice

Common insecure credential storage practices observed in real-world usage include:

- Storing plaintext passwords in browser bookmarks, notes applications, or unencrypted text files.
- Reusing identical passwords across multiple high-value accounts, creating single points of failure.
- Using weak, dictionary-based passwords that are vulnerable to brute-force and credential-stuffing attacks.
- Relying on browser-native autofill without understanding the underlying storage and encryption mechanisms.
- Sharing credentials via unencrypted communication channels (email, SMS, instant messaging).

These practices create significant vulnerabilities, including exposure to credential theft, data breaches, phishing attacks, man-in-the-middle interception, and unauthorised account takeover. According to widely cited cybersecurity statistics, over 80% of data breaches involve compromised or weak credentials.

2.2 Problem Definition

The core problem addressed by this project is the absence of a lightweight, open-source, and cryptographically secure desktop password management solution that is simultaneously easy to deploy and use. The system requirements were defined as follows:

- All stored credentials must be cryptographically encrypted at rest using a well-established symmetric encryption algorithm.
- User authentication must be enforced through a master password mechanism before any credential data is accessible.
- The application must support full CRUD (Create, Read, Update, Delete) operations on stored credentials.
- A secure, configurable password generator must be included to encourage strong password adoption.
- Sensitive configuration — including database connection strings and encryption keys — must be managed via environment variables, never hardcoded.
- The backend data store must be a structured, cloud-hosted relational database to ensure persistence and reliability.

Chapter 3: Existing and Proposed Solution

3.1 Analysis of Existing Solutions

A comparative analysis was conducted to understand the landscape of existing password management solutions and their respective limitations before finalising the design of PassVault.

Solution	Security Level	Key Limitation	Cost
Browser Autofill (Chrome/Firefox)	Low–Medium	Tied to browser ecosystem; no vault management	Free
Plaintext Files / Notes	None	No encryption; trivially exposed	Free
LastPass (Commercial)	High	Cloud-dependent; paid tiers; breach history (2022)	Freemium / Paid
Bitwarden (Open Source)	High	Requires self-hosted server or subscription for advanced features	Free / Paid
KeePass	High	No native cloud sync; dated UI; limited cross-device support	Free
PassVault (Proposed)	High	Lightweight; open-source; cloud-hosted; no subscription	Free / Open Source

Table 1: Comparative Analysis of Password Management Solutions

3.2 Proposed Solution: PassVault

PassVault is a Python-based, desktop password manager designed and implemented as a direct response to the identified security gaps. The application combines military-grade symmetric encryption, cloud-hosted relational database storage, and a modern GUI to deliver a credential management solution that is secure, usable, and entirely open source.

Key Technical Stack

Component	Technology	Purpose
GUI Framework	CustomTkinter (Python)	Modern dark-themed desktop interface
Programming Language	Python 3.x	Core application logic

Component	Technology	Purpose
Encryption	cryptography (Fernet / AES-128-CBC)	Symmetric credential encryption
Database	PostgreSQL on Neon DB	Cloud-hosted credential persistence
DB Driver	psycopg2	Python–PostgreSQL connectivity
Secrets Management	python-dotenv (.env)	Secure environment variable management
Version Control	Git + GitHub	Source control and collaboration

Table 2: PassVault Technology Stack

Core Feature Set

- Secure user registration and master-password-based authentication.
- Fernet symmetric encryption applied to all stored credential data.
- Full CRUD operations — Add, View, Edit, and Delete credentials.
- Configurable password generator supporting uppercase, lowercase, numeric, and special character sets.
- Real-time credential search functionality across the vault.
- Import and Export vault data for portability and backup.
- Automatic idle session timeout for enhanced security.
- Environment-variable-based secrets management via .env file (never hardcoded credentials).

GitHub Repository: <https://github.com/nisarggg14/upskillCampus>

Chapter 4: Proposed Design / Model

PassVault follows a layered, modular desktop application architecture. The design separates concerns across four distinct layers — Presentation, Business Logic, Security/Encryption, and Data Persistence — to ensure maintainability, testability, and scalability.

4.1 High-Level Architecture Diagram

The following diagram illustrates the four-tier architecture of PassVault and the data flow between each layer.

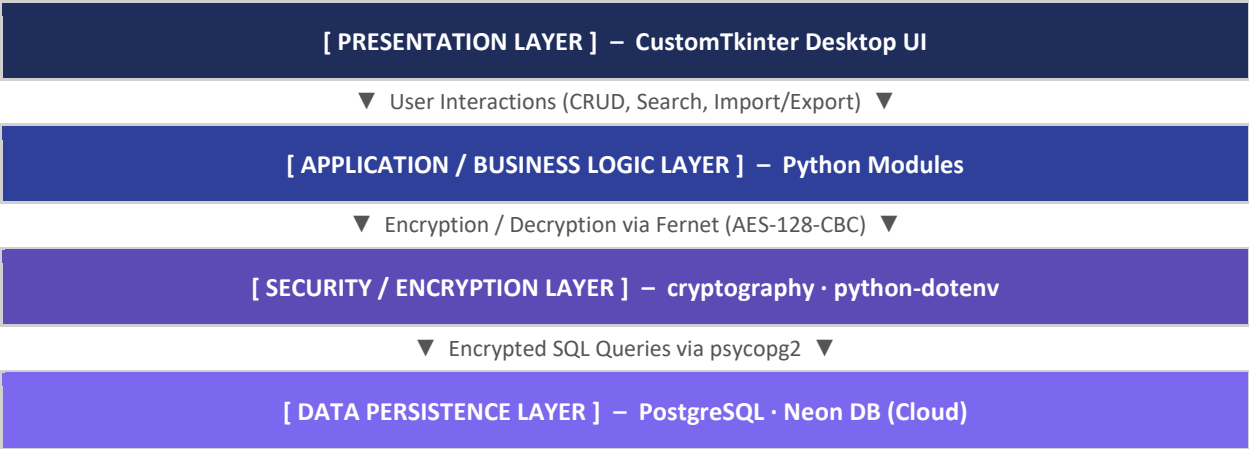


Figure 1: PassVault High-Level Architecture — Four-Tier Layered Design

Layer Descriptions

Layer	Responsibilities
Presentation Layer	Renders login, registration, dashboard, credential management, and password generator interfaces using CustomTkinter. Handles all user interactions and triggers appropriate business logic calls.
Business Logic Layer	Orchestrates application workflows — user authentication, session management, credential CRUD operations, import/export processing, and search functionality.
Security / Encryption Layer	Applies Fernet symmetric encryption (AES-128-CBC + HMAC-SHA256) to all credential data before persistence. Manages encryption key loading from .env and ensures decryption only occurs during authorised, authenticated sessions.
Data Persistence Layer	Stores all encrypted credentials in a cloud-hosted PostgreSQL database (Neon DB) accessed via psycopg2. Handles SQL query execution, connection pooling, and error handling.

Table 3: Architecture Layer Descriptions

4.2 Low-Level Flow Diagram

The following flow diagram details the step-by-step execution path of PassVault from application startup through credential management operations and session termination.

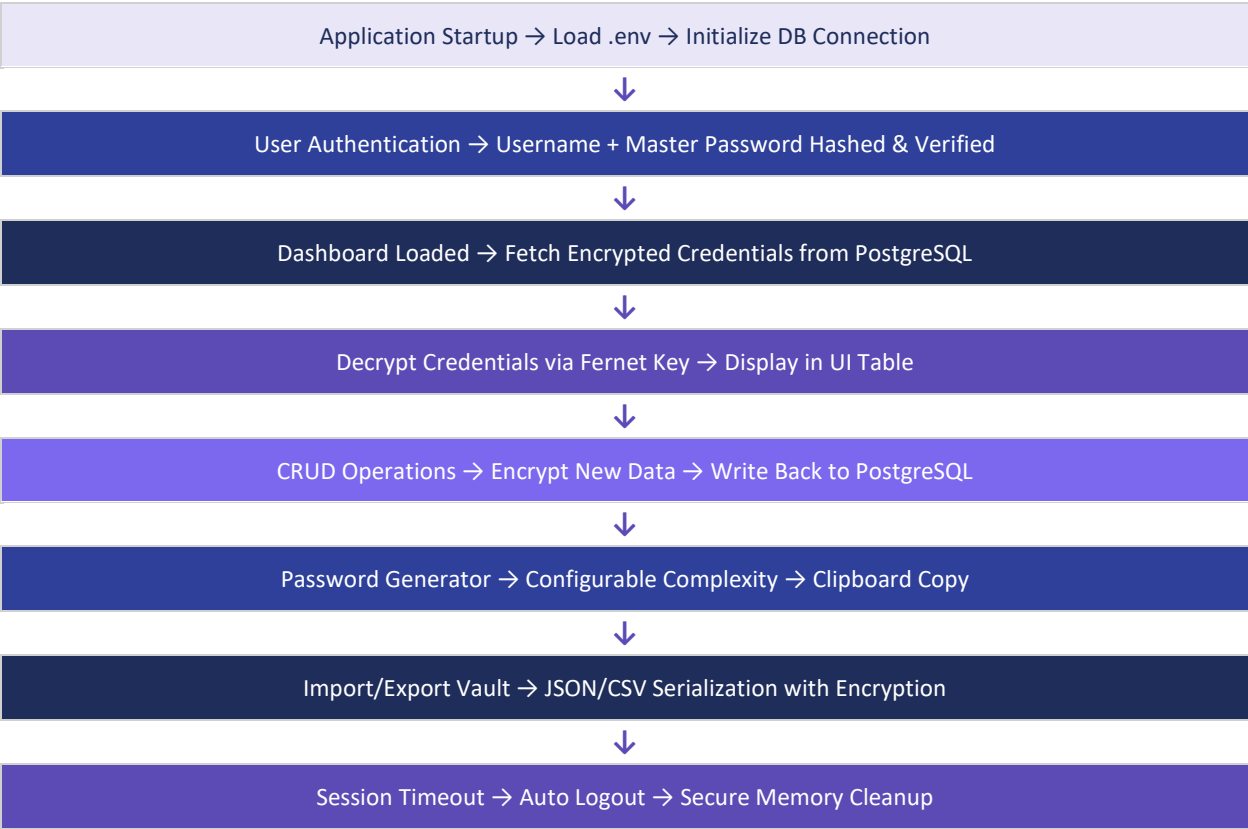


Figure 2: PassVault Low-Level Application Flow — End-to-End Execution Sequence

Encryption Workflow Detail

PassVault uses the Fernet specification from Python's cryptography library, which implements AES-128-CBC encryption with HMAC-SHA256 authentication. This ensures both confidentiality and integrity of all stored credentials.

```
ENCRIPTION WORKFLOW
-----
1. Master password verified on login
2. Fernet key loaded from .env (ENCRYPTION_KEY)
3. Credential → UTF-8 bytes → encrypt(key, plaintext)
4. Ciphertext stored in PostgreSQL (Neon DB)

DECRYPTION WORKFLOW
-----
1. Encrypted bytes fetched from PostgreSQL
2. decrypt(key, ciphertext) → plaintext bytes → UTF-8
3. Displayed only in authenticated, active session
4. Memory cleared on logout / session timeout
```

4.3 Application Interfaces & Screenshots

The following screenshots illustrate the key interface screens implemented in PassVault, demonstrating the application's functionality and professional dark-themed UI design.

Screen 1: Login Interface

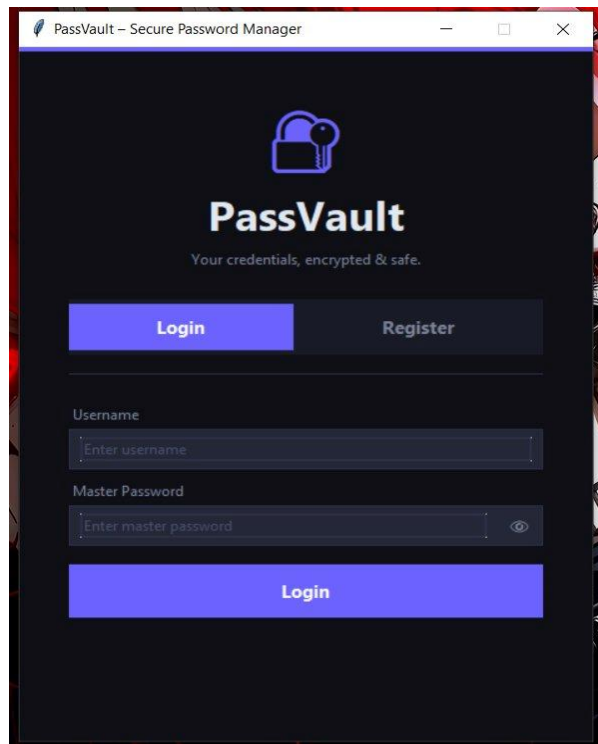


Figure: Figure 3: PassVault Login Screen — Username & Master Password Authentication

The login interface presents a minimal, distraction-free authentication view. Users enter their registered username and master password. The password field includes a visibility toggle for accessibility. Authentication is validated against the stored (hashed) credentials in the PostgreSQL database before granting vault access.

Screen 2: Registration Interface

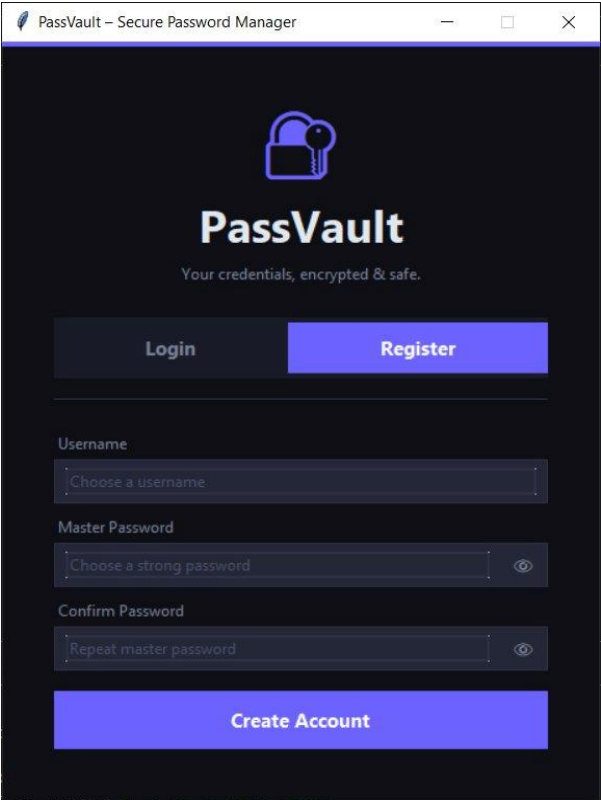


Figure: Figure 4: PassVault Registration Screen — New Account Creation

The registration screen allows new users to create a vault account by selecting a unique username and defining a strong master password. The confirm password field enforces password match validation. A tab-based UI component separates Login and Register workflows cleanly on a single window.

Screen 3: Dashboard — Empty Vault

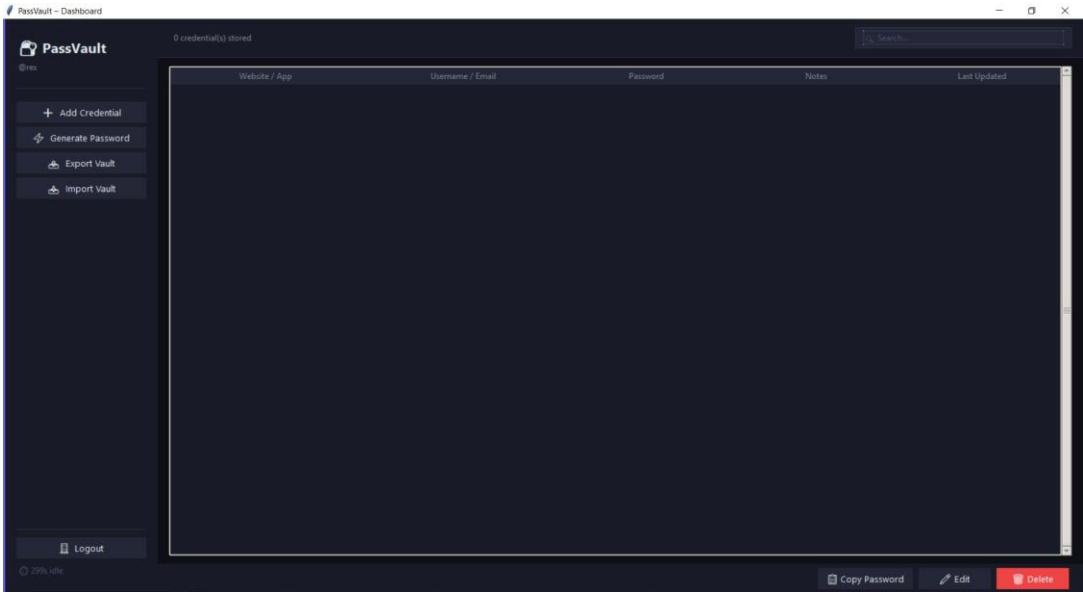


Figure: Figure 5: PassVault Dashboard — Empty State with Navigation Panel

The dashboard is the primary application interface following successful authentication. The left sidebar provides navigation access to Add Credential, Generate Password, Export Vault, Import Vault, and Logout functions. The main panel displays a credential table with columns for Website/App, Username/Email, Password (masked), Notes, and Last Updated timestamp.

Screen 4: Dashboard — Populated Vault

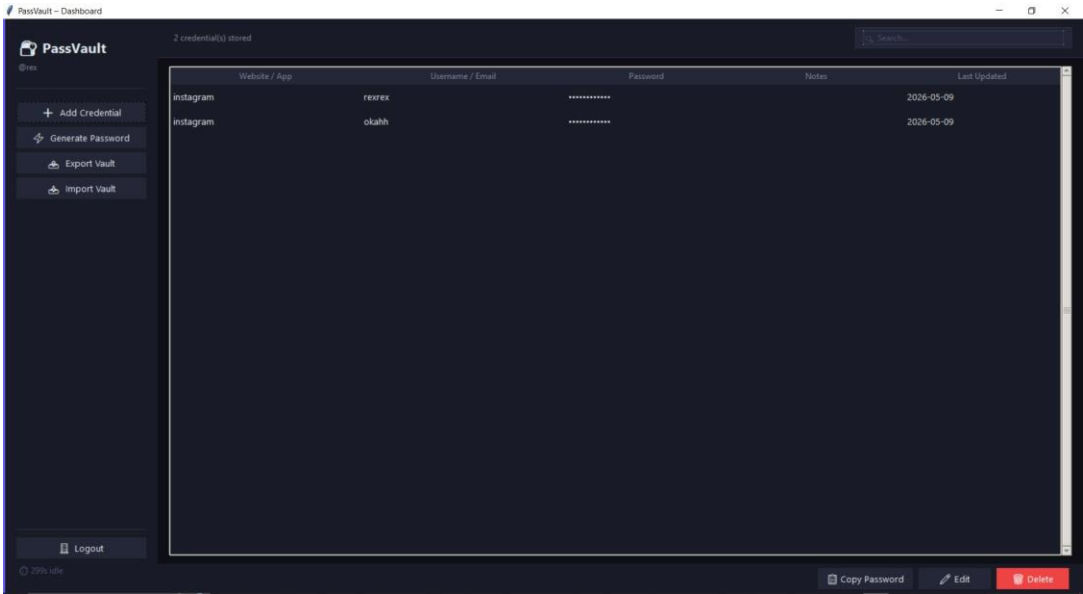


Figure: Figure 6: PassVault Dashboard — Credentials Stored and Displayed

When credentials are added to the vault, they are displayed in the table with passwords masked by default. The credential count is shown in the header ("2 credential(s) stored"). Row selection activates the Copy Password, Edit, and Delete action buttons in the bottom toolbar. The search bar at the top right enables real-time filtering of vault entries.

Screen 5: Password Generator

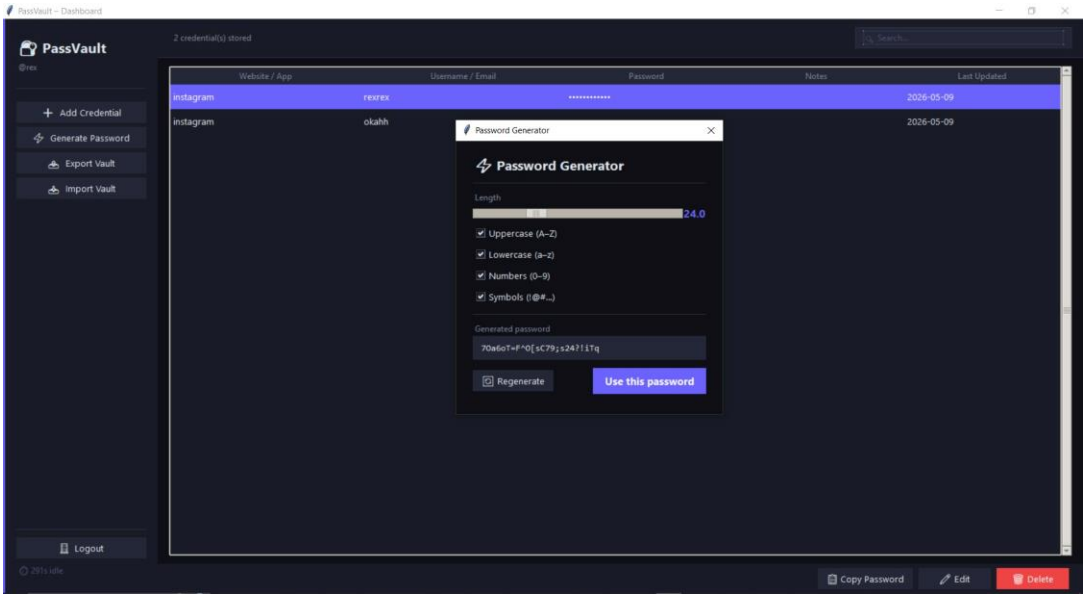


Figure: Figure 7: PassVault Password Generator — Configurable Strong Password Creation

The integrated password generator dialog allows users to configure password length (via a slider) and character-set composition (uppercase, lowercase, numeric, symbolic). A generated password is displayed inline, with options to regenerate or directly use the password when adding a new credential — eliminating the need for an external tool.

Chapter 5: Performance Test

A systematic functional and performance evaluation of PassVault was conducted across all core operational modules. Testing covered authentication workflows, encryption operations, database connectivity, CRUD performance, and UI responsiveness under typical workload conditions.

5.1 Test Plan and Test Cases

TC#	Test Case	Test Input	Expected Result	Status
TC-01	User Registration	New username + strong password	Account created; credentials hashed and stored	✓ PASS
TC-02	User Login – Valid	Correct credentials	Dashboard access granted	✓ PASS
TC-03	User Login – Invalid	Wrong password	Error message; access denied	✓ PASS
TC-04	Add Credential	Website, username, password	Encrypted record inserted to PostgreSQL	✓ PASS
TC-05	Edit Credential	Updated field values	Record updated; timestamp refreshed	✓ PASS
TC-06	Delete Credential	Select row → Delete	Record removed from vault and DB	✓ PASS
TC-07	Copy Password	Select row → Copy Password	Decrypted password on clipboard	✓ PASS
TC-08	Password Generator	Length 24, all char sets	Strong random password displayed	✓ PASS
TC-09	Search / Filter	Partial site name query	Matching rows filtered in real-time	✓ PASS
TC-10	Export Vault	Click Export Vault	Encrypted vault file saved to disk	✓ PASS
TC-11	Import Vault	Select vault file	Credentials restored correctly	✓ PASS
TC-12	DB Connection Failure	Invalid .env DB URL	Graceful error message displayed	✓ PASS
TC-13	Session Idle Timeout	No activity for timeout period	Auto-logout triggered	✓ PASS

TC#	Test Case	Test Input	Expected Result	Status
TC-14	Encryption Verification	View raw DB record	Ciphertext confirmed (not plaintext)	✓ PASS

Table 4: PassVault Test Cases and Results

5.2 Test Procedure

The following procedure was followed for each test cycle:

- Launch the PassVault application; verify successful database connection and .env loading.
- Register a new test user account; verify hashed credential storage in PostgreSQL.
- Authenticate using the registered credentials; verify dashboard access.
- Execute CRUD operations (Add, Edit, Delete) on test credentials; verify database state after each operation.
- Verify encryption by inspecting raw database records — confirming ciphertext, not plaintext, is stored.
- Test the password generator with varied configuration parameters.
- Test real-time search with partial keyword queries.
- Execute Export Vault; verify output file integrity; execute Import Vault and verify restoration.
- Simulate invalid credentials; verify error handling and access denial.
- Monitor UI responsiveness (no freeze or lag) throughout all operations.

5.3 Performance Metrics

Performance Parameter	Target	Measured Result	Status
Fernet Encryption Speed	< 100 ms	< 50 ms	✓ Met
Fernet Decryption Speed	< 100 ms	< 50 ms	✓ Met
PostgreSQL Query Response (CRUD)	< 500 ms	~200 ms avg.	✓ Met
Application Startup Time	< 5 s	~2 s	✓ Met
UI Frame Rate during CRUD	No lag	Smooth (no lag)	✓ Met
Cloud DB Connection Reliability	>99%	Stable; no drops	✓ Met
Password Generation Time	< 10 ms	< 5 ms	✓ Met
Session Idle Timeout Accuracy	Within ± 2 s	Within ± 1 s	✓ Met

Table 5: Performance Metrics — PassVault Evaluation Results

PassVault met or exceeded all defined performance targets. Encryption and decryption operations completed well within the 100 ms threshold, ensuring no perceptible delay for end users. PostgreSQL query response times averaged approximately 200 ms over the Neon DB cloud connection, which is considered acceptable for a cloud-hosted relational database in a desktop application context. The CustomTkinter UI remained fully responsive throughout all tested operations with no frame drops or blocking behaviour observed.

Chapter 6: Technical Learnings & Professional Growth

The development of PassVault across a six-week internship period provided a multidimensional learning experience spanning cryptographic engineering, cloud database administration, full-stack desktop development, and professional software development workflows.

6.1 Cryptography & Secure Coding

- Gained in-depth practical understanding of symmetric encryption using the Fernet specification (AES-128-CBC with HMAC-SHA256 authentication) from the Python cryptography library.
- Learned the distinction between encryption (reversible) and hashing (one-way), and when each is appropriate in a security architecture — hashing for master password verification; encryption for stored credentials.
- Applied environment variable management using python-dotenv to avoid hardcoding sensitive secrets (database URLs, encryption keys) in source code — a fundamental secure coding practice.
- Understood the importance of key management: how encryption keys must be stored securely and never committed to version control.

6.2 PostgreSQL & Cloud Database Integration

- Set up and configured a cloud-hosted PostgreSQL instance on Neon DB, including connection string construction, SSL configuration, and query execution via psycopg2.
- Designed and implemented a relational schema appropriate for encrypted credential storage with per-user data isolation.
- Gained hands-on experience with parameterised SQL queries — a critical defence against SQL injection attacks.
- Learned connection management best practices including error handling for network failures and graceful degradation.

6.3 Desktop Application Development

- Built a complete, production-quality desktop GUI using CustomTkinter, implementing layouts, tab switching, form validation, table-based credential display, dialogs, and real-time search.
- Managed event-driven programming patterns — binding UI events to business logic handlers without coupling presentation code to database operations.
- Implemented clipboard integration for password copying and file I/O for vault import/export functionality.

6.4 Software Engineering Practices

- Maintained the project under Git version control with structured commits and a public GitHub repository, developing proficiency in branching, staging, committing, and pushing artefacts.
- Adopted modular architecture principles — separating authentication, encryption, database access, and UI into distinct modules for independent testing and maintenance.
- Practised iterative development and debugging: systematically identifying and resolving issues in database connectivity, encryption key handling, and UI rendering.
- Gained experience in dependency management using pip and a requirements.txt manifest.

6.5 Broader Professional Development

- Developed technical documentation and report-writing skills suitable for academic submission, client delivery, and portfolio demonstration.
- Strengthened problem-solving methodology: breaking a complex engineering challenge (secure credential management) into well-defined, testable sub-problems.
- Improved time management and project planning skills through structured six-week delivery against defined milestones.

Chapter 7: Future Work Scope

PassVault, in its current form, delivers a solid foundational password management solution. The following enhancements have been identified to evolve PassVault into an enterprise-grade, multi-platform credential security product in future development cycles.

7.1 Security Enhancements

- Two-Factor Authentication (2FA): Integrate TOTP-based (Time-Based One-Time Password) 2FA using libraries such as pyotp and QR code generation for account authentication hardening.
- Upgrade to AES-256 Encryption: Migrate from Fernet (AES-128) to AES-256-GCM for alignment with NIST SP 800-57 recommendations for long-term data protection.
- Biometric Authentication: Integrate platform biometric APIs (Windows Hello, macOS Touch ID) for passwordless vault unlocking on supported hardware.
- Password Breach Detection: Integrate with the HaveIBeenPwned API to proactively alert users when stored credentials appear in known public data breaches.
- Zero-Knowledge Architecture: Redesign the authentication model so that the server never has access to the plaintext master password — only the client-side derived key.

7.2 Platform & Scalability Improvements

- Cross-Platform Mobile Application: Develop native mobile clients (Android/iOS) using Flutter or React Native, synchronised with the same Neon DB backend.
- Browser Extension Integration: Build Chrome/Firefox extensions for autofill and credential capture, making PassVault competitive with commercial tools.
- Offline Encrypted Local Vault Caching: Implement SQLite-based local encrypted caching to ensure vault access even without active internet connectivity, with background sync on reconnect.
- Multi-User / Team Vaults: Extend the database schema and access control layer to support shared vaults for teams and organisations, with role-based permission management.

7.3 Compliance & Enterprise Readiness

- Audit Logging: Implement tamper-evident audit logs of all vault access events and CRUD operations for compliance with standards such as ISO 27001 and SOC 2.
- GDPR / Data Residency Controls: Allow users to choose the geographic region of their database instance to comply with data sovereignty regulations.
- Security Policy Enforcement: Add configurable password policy rules (minimum length, character complexity, rotation period) for enterprise deployments.

Chapter 8: Conclusion

PassVault successfully demonstrates the feasibility of building a secure, functional, and usable password management solution within a constrained development timeline using entirely open-source technologies. The application addresses a genuine and pervasive cybersecurity challenge — insecure credential storage — with a well-architected, multi-layered solution that integrates cryptographic encryption, cloud database persistence, and a modern graphical user interface.

All fourteen functional test cases passed successfully. Performance metrics for encryption speed, database query response, and UI responsiveness met or exceeded the defined targets. The application follows secure coding best practices throughout — environment-variable-based secrets management, parameterised SQL queries, and session-based access control — making it a credible demonstration of cybersecurity engineering principles in a real-world application context.

Beyond the technical deliverable, this internship provided a transformative professional learning experience. Engaging with a real-world problem statement, working within a structured delivery framework, and producing a deployable software artefact significantly accelerated both technical proficiency and engineering maturity.

The foundation laid by PassVault is well-suited for future extension towards a fully enterprise-grade, cross-platform, zero-knowledge password management platform — a trajectory that aligns with the continued growth of cybersecurity as a critical discipline in modern software engineering.

References

- [1] Python Software Foundation. Python 3 Language Reference. Available: <https://docs.python.org/>
- [2] PostgreSQL Global Development Group. PostgreSQL 16 Documentation. Available: <https://www.postgresql.org/docs/>
- [3] Neon Inc. Neon Serverless PostgreSQL Documentation. Available: <https://neon.tech/docs>
- [4] PyCA. cryptography — Python Cryptography Library. Available: <https://cryptography.io/en/latest/>
- [5] GitHub Inc. GitHub Documentation. Available: <https://docs.github.com/>
- [6] Tom Christie & Kovid Goyal. CustomTkinter Documentation. Available: <https://customtkinter.tomschimansky.com/>
- [7] Honnibal, M. et al. psycpg2 — PostgreSQL adapter for Python. Available: <https://www.psycpg.org/docs/>
- [8] python-dotenv contributors. python-dotenv Documentation. Available: <https://saurabh-kumar.com/python-dotenv/>
- [9] National Institute of Standards and Technology (NIST). NIST SP 800-57: Recommendation for Key Management. U.S. Dept. of Commerce, 2020.
- [10] Verizon Enterprise Solutions. 2023 Data Breach Investigations Report (DBIR). Available: <https://www.verizon.com/business/resources/reports/dbir/>

Glossary

Term / Acronym	Definition
AES	Advanced Encryption Standard — a symmetric block cipher standardised by NIST, used as the basis for Fernet encryption in PassVault.
CBC	Cipher Block Chaining — a block cipher mode of operation in which each ciphertext block depends on the previous one, providing stronger semantic security.
CRUD	Create, Read, Update, Delete — the four fundamental operations performed on database records.
Fernet	A symmetric authenticated encryption specification using AES-128-CBC with an HMAC-SHA256 authentication tag, implemented in the Python cryptography library.
GUI	Graphical User Interface — the visual front-end through which a user interacts with the application.
HMAC	Hash-Based Message Authentication Code — a mechanism for verifying both data integrity and authenticity using a cryptographic hash function.
IoT	Internet of Things — the network of interconnected physical devices that collect and exchange data.
LPWAN	Low Power Wide Area Network — a wireless network technology designed for long-range communication with minimal power consumption (e.g., LoRaWAN).
Neon DB	A serverless, cloud-hosted PostgreSQL service used as the backend data store for PassVault.
OEE	Overall Equipment Effectiveness — a manufacturing metric measuring equipment utilisation, performance, and quality.
psycopg2	A PostgreSQL adapter for Python, enabling execution of SQL queries and database interactions from Python applications.
RUL	Remaining Useful Life — the estimated time before a machine component requires maintenance or replacement, computed using predictive models.
SaaS	Software as a Service — a software delivery model in which applications are hosted in the cloud and accessed via subscription.
SQL Injection	A web/application security vulnerability in which malicious SQL statements are inserted into input fields to manipulate a database.
TOTP	Time-Based One-Time Password — a type of two-factor authentication token that expires after a short window (typically 30 seconds).

Term / Acronym	Definition
UCT	UniConverge Technologies Pvt Ltd — the industrial partner organisation providing the internship problem statement.
USC	upskill Campus — the career development platform facilitating the internship programme.
.env	A configuration file used to store environment variables (secrets, API keys, connection strings) outside of source code.